Stanford CS224v Course

Agentic AI

Lecture 6

Agentic AI for Graphs and Tables

Shicheng Liu & Monica Lam & Sina Semnani

Lecture Goal

Focus: Semantic Parsing for Graphs and Tables

Agentic Approach

... so You can Apply them to your Project

Outline

- **Data Structures: Graphs and Tables**
  - How to query them with SPARQL & SQL
- Graph QA with SPINACH
  - Challenges and Prior Approaches
  - The Agentic Approach
  - Dataset and Evaluation

- SQL QA with DataTalk

- Guest Lecture: Big Local News on DataTalk (15 min)





Graphs and Tables

	Relational database	Knowledge graph
Data	Rows and columns	Entities and properties
Relationships	Matching keys across tables	Edges between entities
Query language	SQL	SPARQL for RDF graphs
In this lecture	Campaign finance DataTalk	Wikidata SPINACH

A Relational Database

- Rows are records; columns are attributes with defined types
- Keys identify records and link related tables

restaurants

id	location
r1	Bytes
r2	Kali

reviews

id	restaurant_id	author
review_1	r1	Shicheng
review_2	r1	Sina
review_3	r2	Monica

Expressiveness of Query Languages

- Domain agnostic
- All queries of any domain in knowledge bases
  are compositions of a FEW relational algebra operations
  - Basic: Selection, Projection, Cartesian product (Join),
    Union, Set Difference
  - Extended: Sort, Aggregate Operators (Sum, Max, Avg, …)
- Expressive, succinct, well-defined

Amazing CS Idea

WHERE clause: filter

```
CREATE TABLE restaurants (
    id TEXT PRIMARY KEY,
    location TEXT,
    address TEXT,
    cuisines TEXT[],
    rating NUMERIC(2,1),
...);
```

SELECT * FROM
table [WHERE *filter*]?

```
SELECT * FROM restaurants
WHERE location == "Palo Alto"
AND 'chinese' = ANY (cuisines)
```

Show me Chinese restaurants in Palo Alto

WHERE clause: filter

```
CREATE TABLE restaurants (
    id TEXT PRIMARY KEY,
    location TEXT,
    address TEXT,
    cuisines TEXT[],
    rating NUMERIC(2,1),
...);
```

SELECT * FROM
table [WHERE *filter*]?

```
SELECT * FROM restaurants
WHERE location = "Palo Alto"
AND "Chinese" = ANY(cuisines)
AND rating >= 4.5
```

Show me Chinese restaurants in Palo Alto

with at least 4.5 stars

SELECT clause: Projection

```
CREATE TABLE restaurants (
    id TEXT PRIMARY KEY,
    location TEXT,
    address TEXT,
    cuisines TEXT[],
    rating NUMERIC(2,1),
...);
```

SELECT field+ FROM
table [WHERE *filter*]?

SELECT address FROM restaurants
WHERE location = "Palo Alto"
AND "Chinese" = ANY(cuisines)
AND rating >= 4.5

Show me the address of Chinese restaurants in Palo Alto with at least 4.5 stars

Subquery

```
CREATE TABLE restaurants (
    id TEXT PRIMARY KEY,
    location TEXT,
    address TEXT,
    cuisines TEXT[],
    rating NUMERIC(2,1),
...);
```

```
CREATE TABLE reviews (
    id TEXT PRIMARY KEY,
    restaurant_id TEXT
    REFERENCES restaurants(id),
    author TEXT,
...);
```

Atom filter:
param op value

Subquery filter:
param op
*(SELECT * FROM …)*

SELECT field+
FROM *table* [WHERE *filter*]?

SELECT address FROM restaurants
WHERE location = "Palo Alto"
AND "Chinese" = ANY(cuisines)
AND id IN
(SELECT restaurant_id FROM reviews WHERE author =~ "Bob")

Show me the address of Chinese restaurants in Palo Alto reviewed by Bob

Sorting

```
CREATE TABLE restaurants (
    id TEXT PRIMARY KEY,
    location TEXT,
    address TEXT,
    cuisines TEXT[],
    rating NUMERIC(2,1),
...);
```

```
CREATE TABLE reviews (
    id TEXT PRIMARY KEY,
    restaurant_id TEXT
    REFERENCES restaurants(id),
    author TEXT,
...);
```

SELECT field+ FROM *table* [WHERE *filter*]?
ORDER BY *field* DESC/ASC

```
SELECT address FROM restaurants
WHERE location = "Palo Alto"
AND "Chinese" = ANY(cuisines)
AND id IN
(SELECT restaurant_id FROM reviews WHERE author =~ "Bob" )
ORDER BY rating DESC
```

Show me the address of top-rated Chinese restaurants in Palo Alto reviewed by Bob

Joins

```
CREATE TABLE restaurants (
    id TEXT PRIMARY KEY,
    location TEXT,
    address TEXT,
    cuisines TEXT[],
    rating NUMERIC(2,1),
…);
```

```
CREATE TABLE reviews (
    id TEXT PRIMARY KEY,
    restaurant_id TEXT
    REFERENCES restaurants(id),
    author TEXT,
…);
```

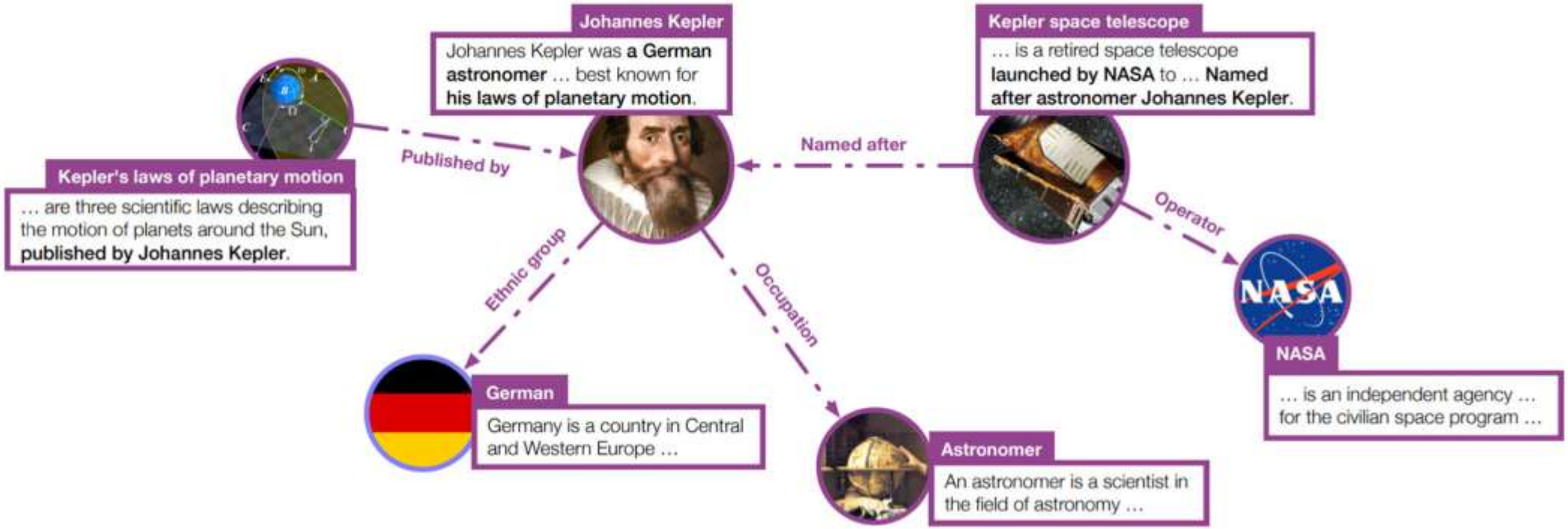
SELECT * FROM
table join *table* [WHERE *filter*]? [on *filter*]?

```
SELECT * FROM restaurants JOIN reviews ON
restaurants.id = reviews.restaurant_id
WHERE location = "Palo Alto"
AND "Chinese" = ANY(cuisines)
ORDER BY rating DESC
```

Show me top-rated Chinese restaurants in Palo Alto with their reviews

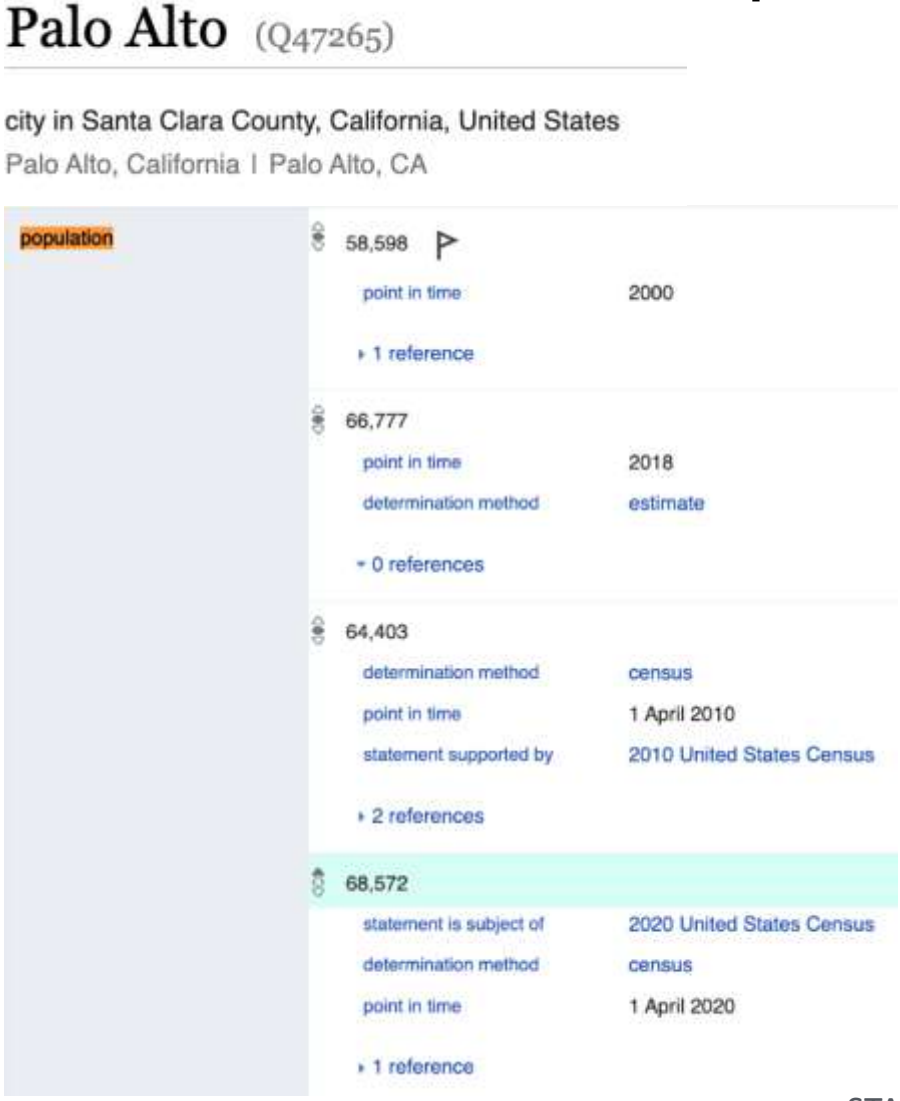
A Knowledge Graph

- Graph databases represent relations among entities as a graph
  - Nodes are entities
  - Edges are relationships; properties describe them



Wikidata: The Largest Live Knowledge Graph

- Stats:
  - 15B facts (1B more triples per year)
  - 100M entities
  - 10K properties (3000 of interest)
  - 25K contributors
- All entities in Wikipedia are in Wikidata
- Wikidata contains many more entities



Wikidata Representation – a RDF graph

- Representation: RDF triples
  - Nodes are entities (represented by unique QIDs)
    - There are many entities with exactly the same name
    - Same ID across all Wikipedia in all languages
  - Edges are properties (represented by unique PIDs)

Query with SPARQL

Who founded Stanford?

SELECT ?x WHERE

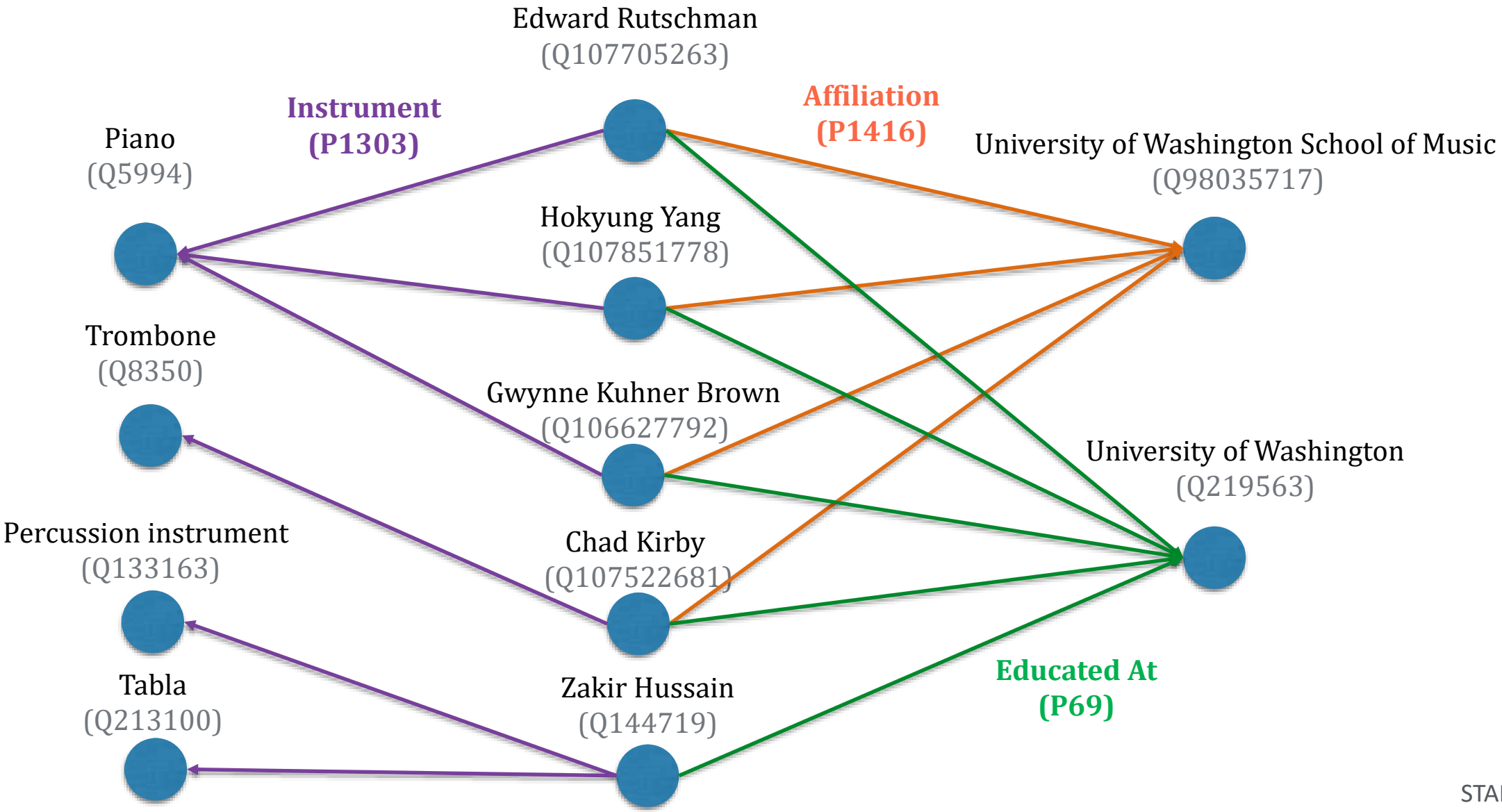
{ wd:Q41506 wdt:P112 ?x. }

Stanford Founded by

Running Example: Music Instruments Played

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

A Subset of the Knowledge Graph in Wikidata



```
>> execute_sparql("""
SELECT ?instrument ?instrumentLabel (COUNT(?student) AS ?count) WHERE {
  ?student wdt:P1303 ?instrument;
    wdt:P1416 wd:Q98035717;
    wdt:P69 wd:Q219563.
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
}
GROUP BY ?instrument ?instrumentLabel
""")
Observation:
   | instrument    | instrumentLabel | count |
   |---------------|-----------------|-------|
   | Q5994         | piano           | 99    |
   | Q1467960      | mbira           | 2     |
   | Q8350         | trombone        | 11    |
   | Q8338         | trumpet         | 8     |
   | Q17172850     | voice           | 32    |
   | ...           | ...             | ...   |
   | Q302497       | mandolin        | 1     |
   | Q187851       | recorder        | 1     |
   | Q185041       | cor anglais     | 1     |
   | Q83509        | piccolo         | 1     |

>> stop()
```

Writing the correct SPARQL query after correcting the mistake

P1303: instrument
P1416: affiliation
P69: educated at

Q98035717:
University of Washington
School of Music

Q219563:
University of Washington

The Power of WikiData

- SPARQL allows relational algebra operations
  across the entire knowledge graph
  - Running example needs: filters, projections, joins, counts …
  - Query optimizations

- Dataset for research in
  Mathematics, **Biology**, **Education**, **Social Sciences**, **Linguistics**, ..

Quiz: Which tables would represent these relationships?

Why Querying Wikidata with SPARQL is Difficult

- **Heterogeneous data:**
  - Entities can have different sets of properties
  - The query must match the properties actually stored
  - Inspect entities and examples before choosing a property
- **Many properties (3000):** Hard to memorize all the properties
- **Many similar properties:**
  - Often unclear which property or entity should be used
  - Questions on locations: "Where did Isaac Newton live?" "Where is Salesforce?"
  - Possible properties – depends on what is available for the node
    - *administrative territorial entity*
    - *residence*, *state*, *country*, *place of birth*, *place of death*
    - *headquarters location*, *location of formation*
- **Need to look at a node's properties to determine the right SPARQL**
- **Queries can be complex**

A natural language interface can greatly expand access!

Outline

- Data Structures: Graphs and Tables
- Graph QA with SPINACH
  - **Challenges and Prior Approaches**
  - The Agentic Approach
  - Dataset and Evaluation





- SQL QA with DataTalk

- Guest Lecture: Big Local News on DataTalk (15 min)

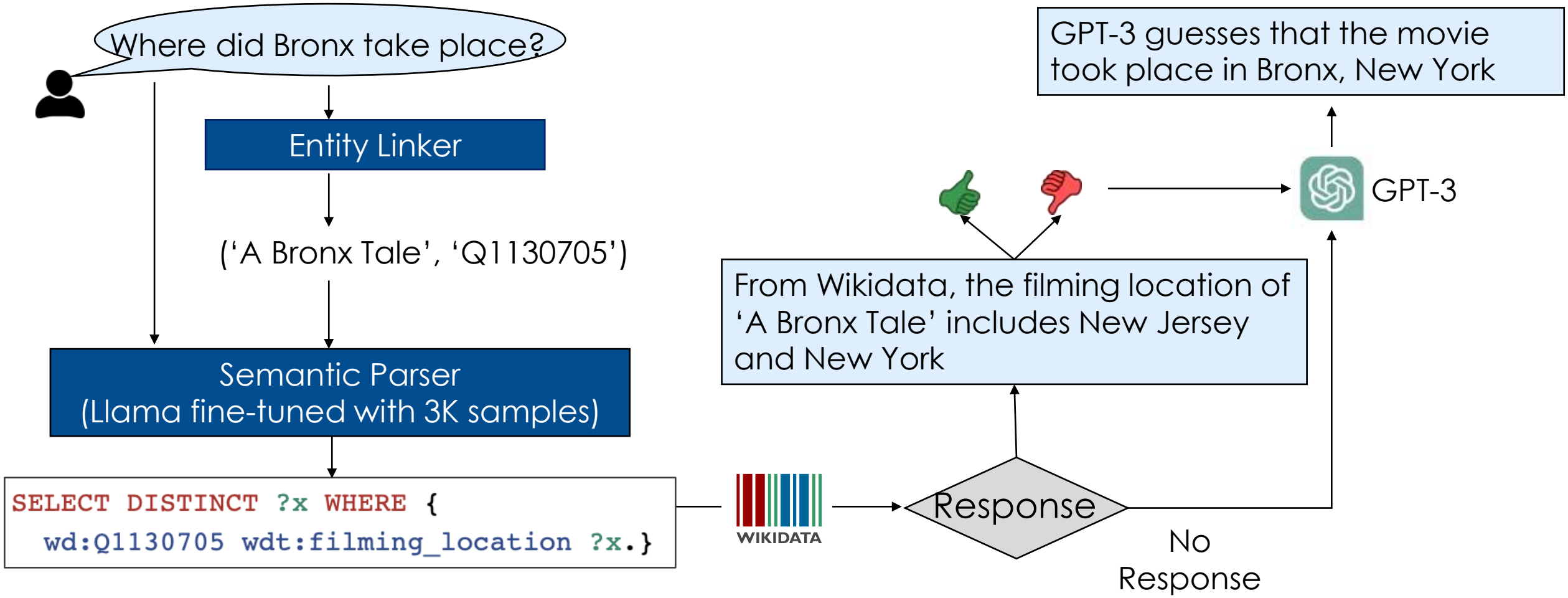
Previous Work

Knowledge Base Query Answering (KBQA)

Semantic Parser: Generating Sparql (Fine-tuning, LLM)

Subgraph Traversal: Traversing the Knowledge Graph without Using SPARQL

1. Semantic Parsing with Fine-Tuning



Silei Xu et al. Fine-tuned LLMs Know More, Hallucinate Less with Few-Shot Sequence-to-Sequence Semantic Parsing over Wikidata.. EMNLP 2023

2. Semantic Parsing with Prompting LLMs

- LLMs know the syntax of SPARQL
  - And just a few PIDs and QIDs of entities
  - Cannot parse “research questions”
    (with many joins, group-bys, ranks, …)

3. Subgraph Retrieval (No SPARQL)

Retrieve a part of the graph based on a question

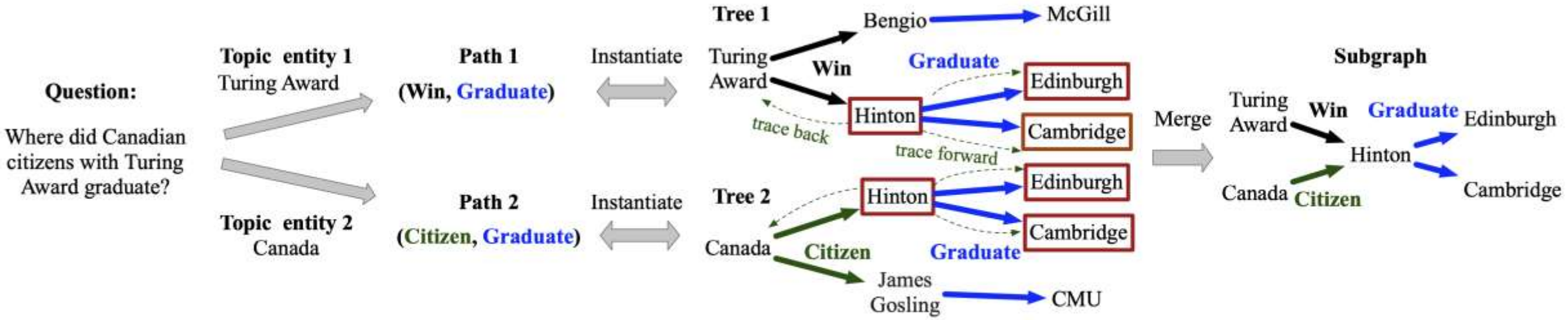


Figure 2: Illustration of the subgraph retrieving process. We expand a path from each topic entity as well as induce a corresponding tree, and then merge the trees from different topic entities to form a unified subgraph.

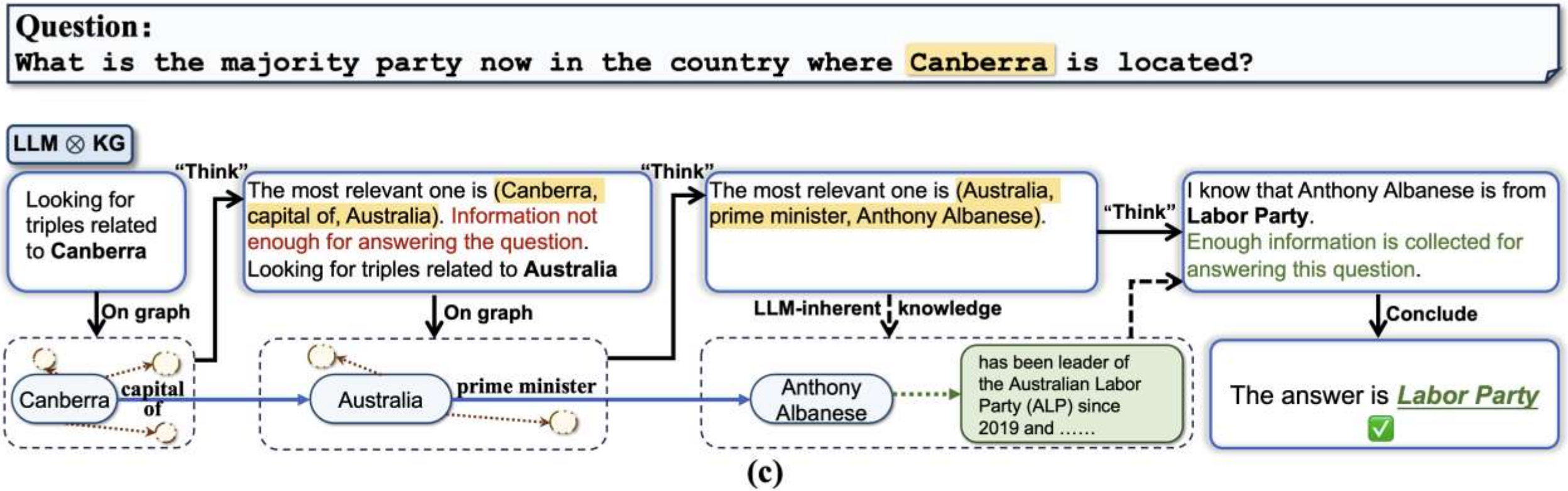
Quiz: Can this find the tallest mountain?

Quiz: Can this provide the full generality of SPARQL

Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering (Zhang et al., ACL 2022)

4. LLM-Based Graph Exploration

Explore a sub-graph by walking the graph one edge at a time



Quiz: Can this provide the full generality of SPARQL

Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph, Sun et al, ICLR 2024

Outline

- Data Structures: Graphs and Tables
- Graph QA with SPINACH
  - Challenges and Prior Approaches
  - **The Agentic Approach**
  - Dataset and Evaluation





- SQL QA with DataTalk

- Guest Lecture: Big Local News on DataTalk (15 min)

Agentic AI

An AI system

that can make decisions and take actions on its own

to achieve a goal



SPINACH:

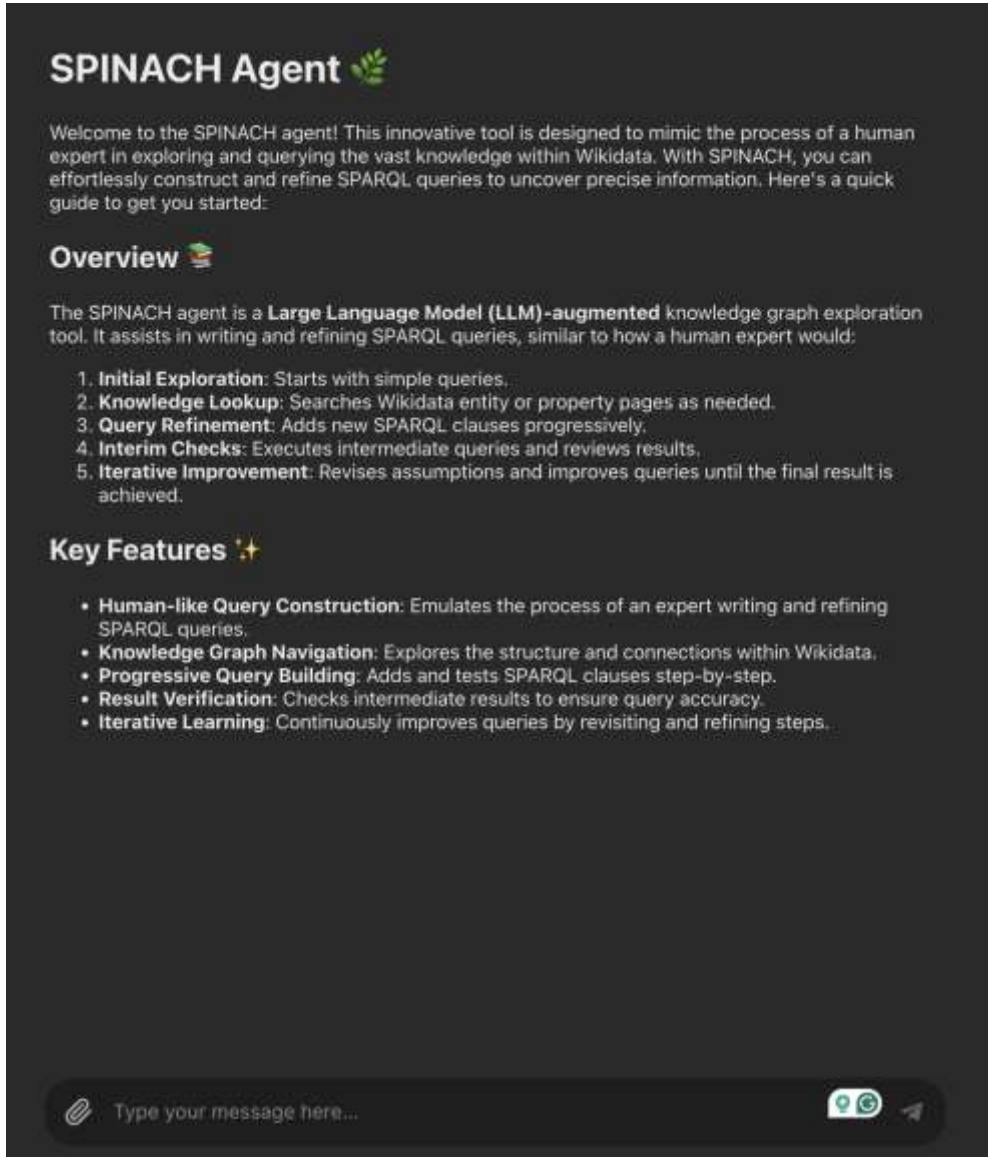
SPARQL-Based Information Navigation for Challenging Real-World Questions

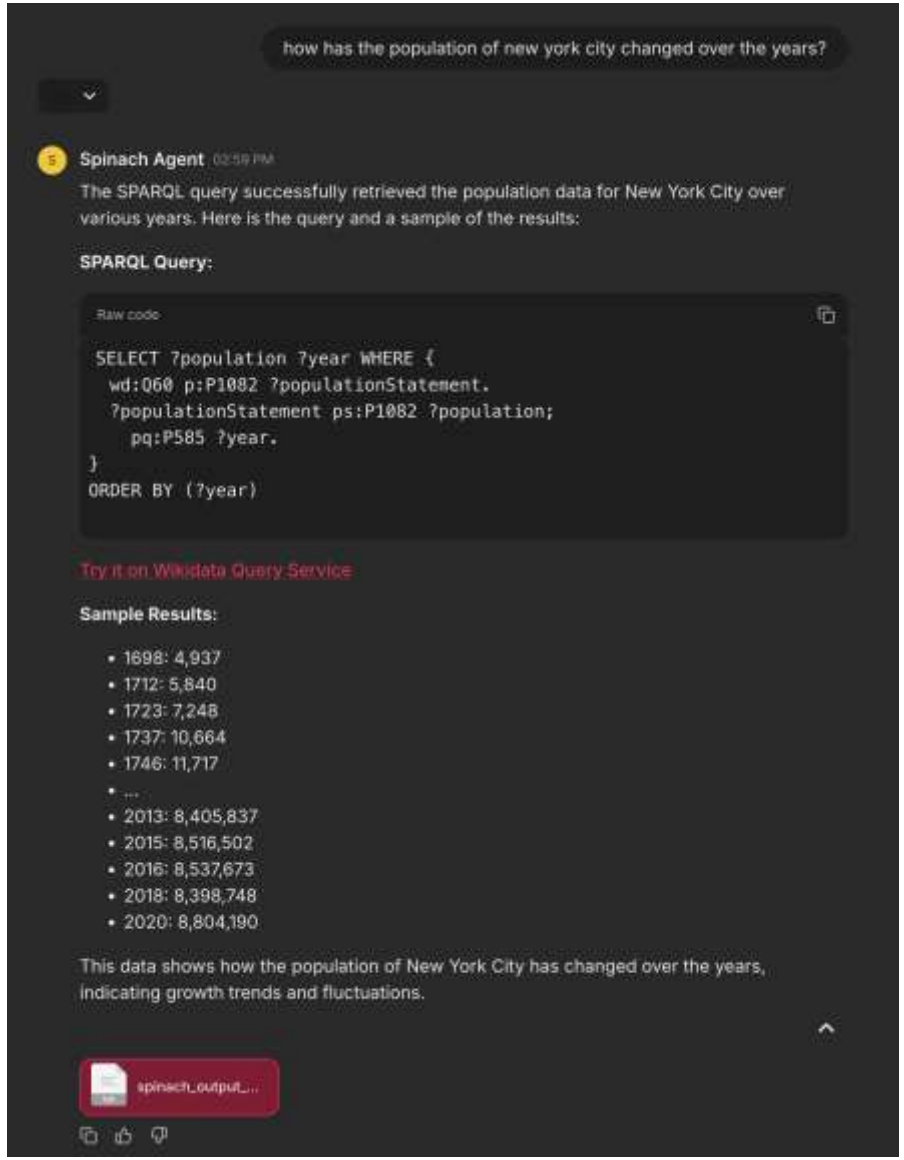
Shicheng Liu* Sina J. Semnani*
Harold Triedman[1] Jialiang Xu Isaac Dan Zhao Monica S. Lam
Stanford University

* Equal contribution
[1] Cornell Tech; Work conducted while at the Wikimedia Foundation

EMNLP 2024

Https://spinach.genie.stanford.edu







Combines the Best of Semantic Parsing (SPARQL), Subgraph Retrieval, & LLMs

- Semantic parsing to SPARQL:
  - Expressiveness, implementation/query optimizations
- Graph retrieval: exam actual properties in the data
- LLM agentic approach combines the 2 above
  - If human experts can do it: we can do it!
  - Automate the human expert's approach
  - Humans don't memorize the nodes and properties

  → we don't need fine-tuning! Just use In-Context Learning

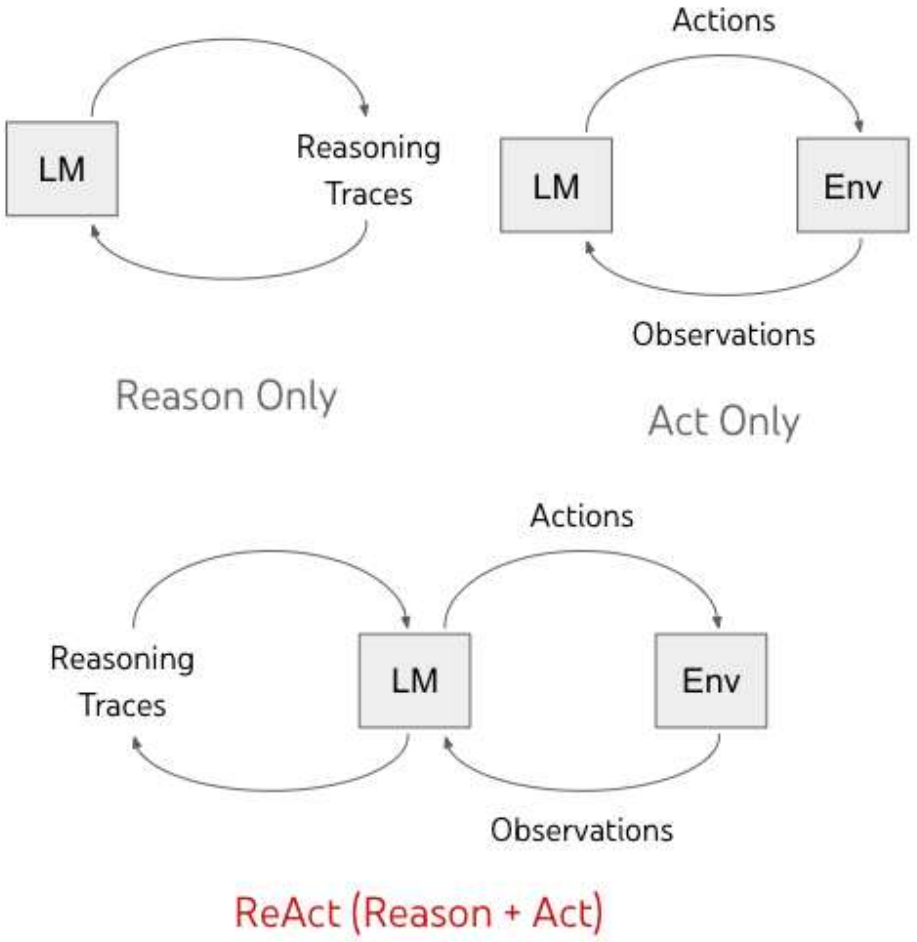
How a Human Expert Write a SPARQL Query

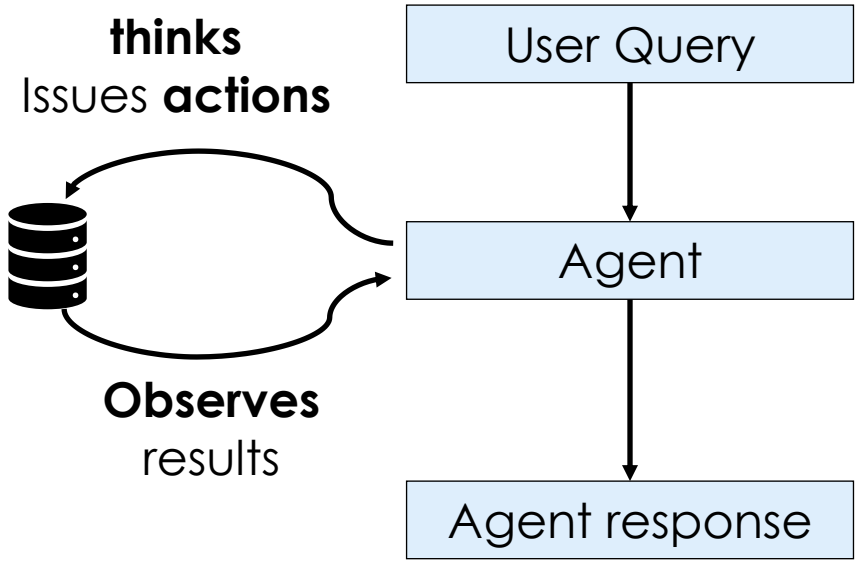
1. Start by writing **simple SPARQL queries**;
2. **Look up** Wikidata **entity or property pages** when needed
   - To understand the structure of the knowledge graph
   - Check what properties exist for a node
3. **Add new SPARQL clauses** to build towards the final SPARQL

LLM Agent weaves together

- Knowledge inquiry
- Query writing
- Execution and evaluation of results (subgraph retrieval)

SPINACH: An Agentic Approach





The same think–act–observe loop powers coding agents such as **Claude Code** and **Codex**.

ReAct: Synergizing Reasoning and Acting in Language Models, Yao et al, ICLR 2023

Actions Useful for Writing SPARQL

Provided by Wikidata

search_wikidata(string): http://wikidata.org (search bar):
text → returns QIDs and PIDs

get_wikidata_entry(QID): https://www.wikidata.org/wiki/QID:<QID>
QID → Wikidata page for entity

get_property_examples(PID): https://www.wikidata.org/wiki/Property:<PID>:
PID → examples of how property PID is used

execute_sparql(SPARQL): https://query.wikidata.org:
SPARQL → result

Running Example

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

Action: **search_wikidata**

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

```
search_wikidata("musical instrument")
```

```
- musical instrument (Q34379)
  device created or adapted to make musical sounds
- heraldic musical instrument (Q56877088)
  category of heraldic charges
- Musical instrument (Q102413357)
  Oil of canvas by Alla Grigoryan
- Musical Instrument (Q102413736)
  painting by Musical instrument
```

This step performs Named Entity Disambiguation (NED) with LLM!

Action: **search_wikidata**

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

```
search_wikidata("affiliation")
```

affiliation (P1416)

organization that a person or organization is affiliated with (not necessarily member of or employed by)

27 statements, 0 sitelinks - 18:16, 28 October 2024

affiliation string (P6424)

qualifier to provide the published string form of affiliation attached to an author

14 statements, 0 sitelinks - 16:31, 1 October 2024

Action: `get_wikidata_entry`

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

```
get_wikidata_entry
("Q98035717")
```

University of Washington School of Music (Q98035717)

```
{
  instance of (P31): {
    faculty (Q180958): {
      Qualifiers: [
        {
          of (P642): University of Washington (Q219563)
        }
      ]
    },
    music school (Q1021290): {}
  },
  field of work (P101): [
    higher education (Q136822),
    postgraduate education (Q141212),
    musicology (Q164204),
    music (Q638),
    research (Q42240),
    ethnomusicology (Q208365)
  ],
  located in the administrative territorial entity (P131): Seattle (Q5083),
  location (P276): Music Building (Q98690890),
  has subsidiary (P355): University of Washington Laboratory for Music
      Cognition, Culture & Learning (Q101157561),
  part of (P361): University of Washington College of Arts and Sciences
      (Q7896566),
  has part(s) (P527): University of Washington Laboratory for Music
      Cognition, Culture & Learning (Q101157561),
  parent organization (P749): University of Washington College of Arts and
      Sciences (Q7896566)
}
```

Action: **get_property_examples**

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

```
get_property_examples("P1416")
```

```
Affiliation (P1416)

Raoul Bott -> Institute for Advanced Study
Hannah Sipe -> University of Washington School
of Environmental and Forest Sciences
```

Action: **execute_sparql**

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

```
execute_sparql("""
SELECT ?instrument ?instrumentLabel (COUNT(?student) AS ?count) WHERE {
  ?student wdt:P1303 ?instrument;
    wdt:P1416 wd:Q98035717;
    wdt:P69 wd:Q219563.
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
}
GROUP BY ?instrument ?instrumentLabel
""")
```

instrument	instrumentLabel	count
Q5994	piano	99
Q1467960	mbira	2
Q8350	trombone	11
Q8338	trumpet	8
Q17172850	voice	32
...	...	...
Q302497	mandolin	1
Q187851	recorder	1
Q185041	cor anglais	1
Q83509	piccolo	1



The SPINACH Agent

- Imitates what the user does with an agentic approach
- Uses the full expressiveness of SPARQL for exploration

- For N steps:
  - Given the history of agent actions,
    - Prompt LLM to generate a **thought** and an **action**
    - Execute an action against the KG
    - Add an **observation** to the history

Main agent code available in this file,
implemented with LangGraph (part of LangChain) in Python

Zero-Shot LLM Policy Prompt

instruction

Your task is to write a Wikidata SPARQL query to answer the given question. Follow a step-by-step process:

1. Start by constructing very simple fragments of the SPARQL query.
2. Execute each fragment to verify its correctness. Adjust as needed based on your observations.
3. Confirm all your assumptions about the structure of Wikidata before proceeding.
4. Gradually build the complete SPARQL query by adding one piece at a time.
5. Do NOT repeat the same action, as the results will be the same.
6. The question is guaranteed to have an answer in Wikidata, so continue until you find it.
7. If the user is asking a True/False question with only one answer, use ASK WHERE to fetch a True/False answer at the very end.
8. In the final SPARQL projections, do not only ask for labels.
   Ask for the actual entities whenever needed (e.g. instead of doing `SELECT xLabel`, do `SELECT x`).
9. If the final result was contained in last round's `get_wikidata_entry` and you are ready to stop,
   use `execute_sparql` and generate a SPARQL to retrieve that results.

Form exactly one "Thought" and perform exactly one "Action", then wait for the "Observation".

Possible actions are:

- get_wikidata_entry(QID): Retrieves all outgoing edges (linked entities, properties, and qualifrs) of a specified Wikidata entity using its QID.
- search_wikidata(string): Searches Wikidata for entities or properties matching the given string.
- get_property_examples(PID): Provides a few examples demonstrating the use of the specified property (PID) in Wikidata.
- execute_sparql(SPARQL): Runs a SPARQL query on Wikidata and returns a truncated result set for brevity.
- stop(): Marks the last executed SPARQL query as the final answer and ends the process.

```
# input
{% for i in range(0, conversation_history|length) %}
User Question: {{ conversation_history[i]["question"] }}
{% for j in range(0, conversation_history[i]["action_history"]|length) %}
{{ conversation_history[i]["action_history"][j] }}
{% endfor %}
--
{% endfor %}

User Question: {{ question }}

{% if action_history %}
{% for i in range(0, action_history|length) %}

{{ action_history[i] }}
{% endfor %}
{% endif %}

Output one "Thought" and one "Action":
```

Templates written in [jinja syntax]

Problem: Looping

- SPINACH needs to explore different queries during iteration
  - Run policy prompt with temperature 1.0
- LLM can fall into a loop of executing the same action

```
execute_sparql("SELECT ?x WHERE {wd:Q1 wdt:P1 ?x.}")
Did not find any results
execute_sparql("SELECT ?x WHERE {wd:Q1 wdt:P1 ?x.}")
Did not find any results
execute_sparql("SELECT ?x WHERE {wd:Q1 wdt:P1 ?x.}")
…
```

Quiz: Why would this happen?
Quiz: How to solve this?

Solution: Detecting & Stopping Loops

- If repeated actions are found,
  - Agent resets exploration state to the one before the repetition
  - Continues from there

- We cap number of actions by either:
  - 15 actions after taking “rollbacks” into account
  - a total of 30 actions including “rollbacks”

The SPINACH Agent in Action (1)

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

Thought: First, I need to identify the properties and entities related to people affiliated with the University of Washington School of Music ...

>> **search_wikidata**("University of Washington School of Music")

Observation: University of Washington School of Music (Q98035717)

[3 more time steps omitted]

>> **search_wikidata**("musical instrument")

Looking up relevant PIDs and QIDs

The SPINACH Agent in Action (2)

Question: What are the musical instruments played by people who are affiliated with the University of Washington School of Music and have been educated at the University of Washington, and how many people play each instrument?

```
>> execute_sparql("""
SELECT ?instrument (COUNT(*) AS ?count) WHERE {
  ?person wdt:P69 wd:Q1063349;
    wdt:P1416 wd:Q98035717;
    wdt:P1303 ?instrument.
}
GROUP BY ?instrument
""")
Obesrvation: Empty result.
```

Attempting to directly write the SPARQL fails due to the hallucinated Q1063349

The SPINACH Agent in Action (3)

```
Thought: Since my previous queries returned empty results, I need to
verify if there are any individuals in Wikidata who have been educated at
the University of Washington. This will help me confirm if I am using the
correct properties and entities.
>> get_wikidata_entry("Q1063349")
Observation:
  Spasskoye (Q1063349) {
    country (P17): Russia (Q159),
    ... more entries ...
  }

Thought: It seems like there is an issue with the QID I used for the
University of Washington. I should search for the correct QID.
>> search_wikidata("University of Washington")
Observation:
  - University of Washington (Q219563)
    public research university in Seattle, Washington, United States
```

Attempting to debug the issue by running simpler SPARQL queries

```
>> execute_sparql("""
SELECT ?instrument ?instrumentLabel (COUNT(?student) AS ?count) WHERE {
  ?student wdt:P1303 ?instrument;
    wdt:P1416 wd:Q98035717;
    wdt:P69 wd:Q219563.
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
}
GROUP BY ?instrument ?instrumentLabel
""")
Observation:
   | instrument    | instrumentLabel | count |
   |---------------|-----------------|-------|
   | Q5994         | piano           | 99    |
   | Q1467960      | mbira           | 2     |
   | Q8350         | trombone        | 11    |
   | Q8338         | trumpet         | 8     |
   | Q17172850     | voice           | 32    |
   | ...           | ...             | ...   |
   | Q302497       | mandolin        | 1     |
   | Q187851       | recorder        | 1     |
   | Q185041       | cor anglais     | 1     |
   | Q83509        | piccolo         | 1     |

>> stop()
```

Writing the correct SPARQL query after correcting the mistake

P1303: instrument
P1416: affiliation
P69: educated at

Q98035717:
University of Washington
School of Music

Q219563:
University of Washington

Outline

- Data Structures: Graphs and Tables
- Graph QA with SPINACH
  - Challenges and Prior Approaches
  - The Agentic Approach
  - **Dataset and Evaluation**





- SQL QA with DataTalk

- Guest Lecture: Big Local News on DataTalk (15 min)

KBQA Data Sets (Crowdsourcing)

- **Datasets with natural questions originally collected through search engines or crowdsourcing**

  ← Quiz: Is this OK?

  - WebQuestionSP (Yih et al., 2016)
  - QALD datasets (Usbeck et al., 2017, 2018, 2023; Perevalov et al., 2022)
  - RuBQ (Korablinov and Braslavski, 2020)
  - SimpleQuestions (Bordes et al., 2015)

Simple Queries

KBQA Data Sets (Synthesized)

- **Datasets with synthetically generated logical forms & questions**
  - ComplexWebQuestions (Talmor and Berant, 2018)
  - GrailQA (Gu et al., 2021)
  - KQA Pro (Cao et al., 2022a)
  - CFQ (Keysers et al., 2020)
  - CWQ (Talmor and Berant, 2018)
  - LC-QuAD2 (Dubey et al., 2019)

Limited NL variety
& Unique query patterns

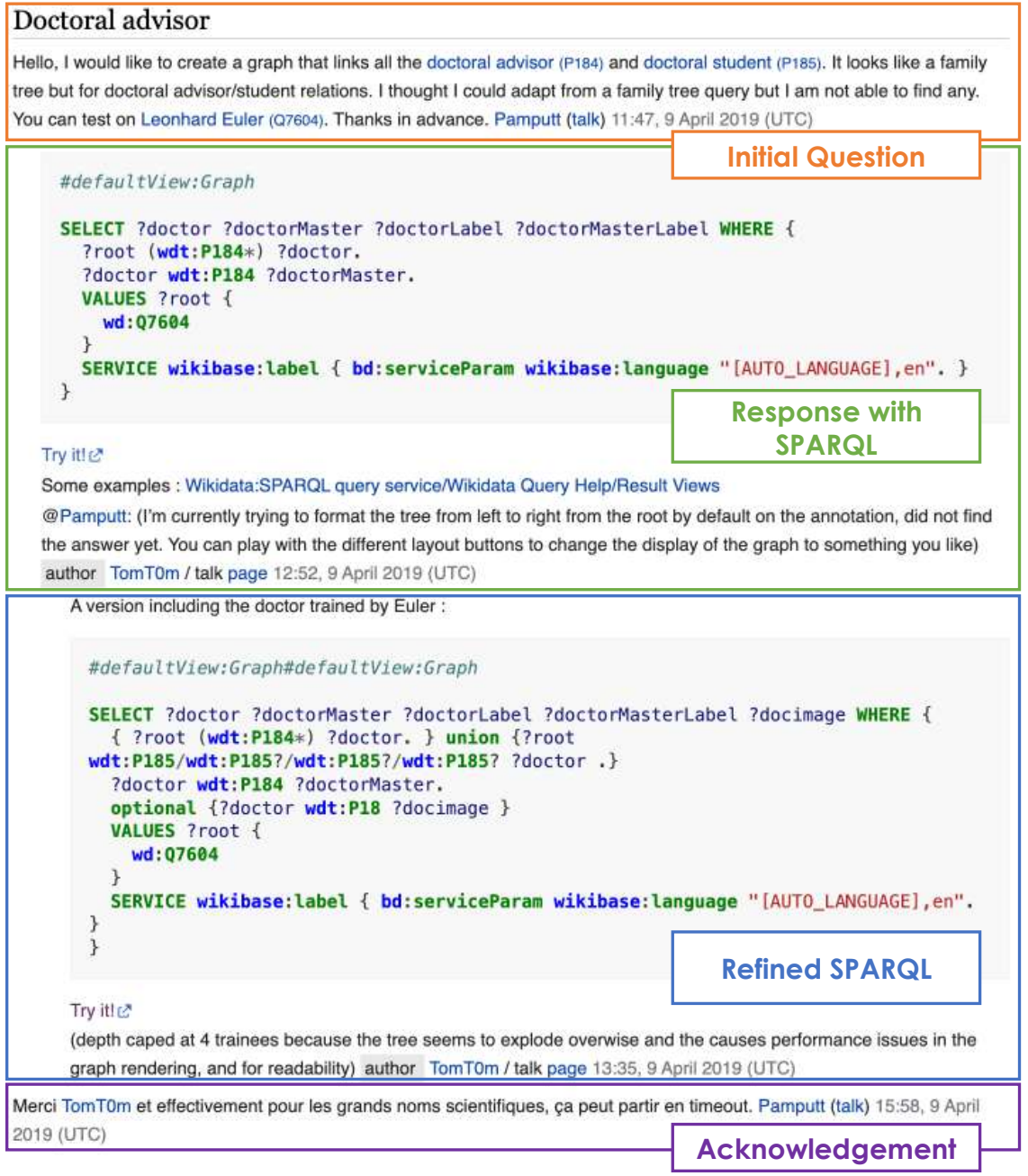
Dataset	Size	UQPs	UQPs / Size
GrailQA (train+dev)	51100	116	0.227%
KQA-Pro	117970	1689	1.432%
CWQ	34689	402	1.159%
WikiWebQuestions	4316	176	0.041%
SPINACH	320	320	100.0%

Table 6: Comparison of Unique Query Patterns (UQPs) in SPINACH and prior works.

Wikidata SPARQL Forum

https://m.wikidata.org/wiki/Wikidata:Request_a_query

- To help Wikidata users write SPARQL queries
- People exchange conversations on how to write SPARQLs
- The queries are real, but difficult



The SPINACH Dataset

From discussions on Wikidata Request Query forum (July 2016 – May 2024)

- **From the natural (real) SPARQL, we annotate its corresponding English question**
  - Disambiguate entities and properties
  - Natural verbalizations
  - Accurately capturing optional clauses and projections
- **We removed these categories of clauses:**
  - Wikimedia presentation queries
  - Questions on complex SPARQL code
  - Queries obscured by optimizations
  - Formatting clauses
- 155 validation examples and 165 test examples

← Quiz: These numbers seem small. Are they enough?

SPINACH dataset:
natural questions + complex logical forms

	Avg. Clauses	Avg. Projs	Avg. Rels	Avg. Subjs	Avg. Preds	Avg. Objs	Avg. Lits
Natural questions w/ annotated logical forms							
WikiWebQuestion (Xu et al., 2023)	2.63	1	1.53	1.25	1.52	1.53	0.04
QALD-9 Plus (Perevalov et al., 2022)	3.14	1	1.77	1.26	1.70	1.78	0.05
QALD-10 (Usbeck et al., 2023)	2.38	1	1.27	1.19	1.17	1.32	0.05
RuBQ (Korablinov and Braslavski, 2020)	2.17	1	1.12	1.03	1.11	1.07	0.01
SimpleQuestionsWikidata (Diefenbach et al., 2017b)	2.00	1	1.00	1.00	1.00	1.00	0.00
Synthetic logical forms w/ synthetic or paraphrased questions							
CWQ (Talmor and Berant, 2018)	5.19	1	2.80	1.87	2.62	3.38	0.11
GrailQA (Gu et al., 2021)	7.10	1	3.02	1.97	2.43	3.90	0.08
KQA Pro (Cao et al., 2022a)	6.34	1	5.01	2.77	3.94	2.43	2.37
MCWQ (Cui et al., 2022)	6.34	1	5.09	2.67	3.53	3.37	0.00
LC-QuAD-2 (Dubey et al., 2019)	3.65	1	2.07	1.51	2.05	2.07	0.22
Natural logical form w/ annotated questions							
SPINACH **(Ours)**	8.89	2.50	4.03	1.76	3.55	4.53	0.46

Results on the SPINACH Dataset

See calculation of F1 for tables in the Appendix of this lecture

Fine-tuned model ->

SOTA. Ask LLM to walk the graph ->

	Dev		Test	
	EM	F1	EM	F1
Direct GPT-4o Question Answering	0.0	3.9	0.0	4.0
GPT-4o Generating SPARQL	1.3	5.4	0.6	3.9
Fine-tuned WikiSP (Xu et al., 2023)	1.3	3.5	1.2	7.1
0-shot ToG (GPT-4) (Sun et al., 2024a)	3.9	9.8	1.8	7.2
0-shot SPINACH agent (GPT-4o) **(Ours)**	**21.4**	**46.4**	**16.4**	**45.3**

SPINACH agent achieves considerable gain over prior approaches!

Fine-tuned LLMs Know More, Hallucinate Less with Few-Shot Sequence-to-Sequence Semantic Parsing over Wikidata, Xu et al, EMNLP 2023

Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph, Sun et al, ICLR 2023

Results on Other Datasets

	QALD-7 (Task 4) Test		QALD-9 Plus (en) Test		QALD-10 (en) Full Test Set		QALD-10 (en) Subset in ToG		WikiWebQuestions Dev		WikiWebQuestions Test	
	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
STAGG (Yih et al., 2016)	-	19.0	-	-	-	-	-	-	-	-	-	-
GGNN (Sorokin and Gurevych, 2018)	-	21.3	-	-	-	-	-	-	-	-	-	-
LingTeQA (To and Reformat, 2020)	-	34.0	-	-	-	-	-	-	-	-	-	-
Baramiia et al. (2022)	-	-	-	-	-	42.8	-	-	-	-	-	-
Shivashankar et al. (2022)	-	-	-	-	-	49.1	-	-	-	-	-	-
QAnswer (Diefenbach et al., 2017a)	-	40.0	-	44.6	-	57.8	-	-	-	-	-	-
SPARQL-QA (Borroto et al., 2022)	-	-	-	-	-	59.5	-	-	-	-	-	-
Liu et al. (2024)	-	-	-	-	56.5	-	-	-	-	-	-	-
0-shot ToG (GPT-4) (Sun et al., 2024a)	-	-	-	-	-	-	54.7	-	-	-	-	-
Fine-tuned WikiSP (Xu et al., 2023)	38.0	43.6	-	-	-	-	-	-	**75.6**	**76.9**	**65.5**	**71.9**
0-shot SPINACH agent (GPT-4o) **(Ours)**	**62.2**	**74.6**	**58.3**	**71.6**	**63.1**	**69.5**	**64.7**	**72.4**	61.2	72.3	59.9	70.3

Zero-shot ICL (in-context learning) achieves new SOTA on QALD Wikidata datasets

Comes within 1.6 F1 on WikiWebQuestions to WikiSP, fine-tuned on WikiWebQuestions

The Importance of Agent Actions

	EM	F1
SPINACH agent	**21.4**	**46.4**
w/o **get_wikidata_entry**	11.7	36.4
w/o **get_property_examples**	10.4	29.4
w/o **search_wikidata**	4.6	25.3

All actions make meaningful contribution to the agent performance

Live Demo:

https://spinach.genie.stanford.edu/

Code:

https://github.com/stanford-oval/spinach

As a bot on Wikidata:

https://www.wikidata.org/wiki/User:SpinachBot

Error Analysis

40%: **Property-related problems**:
Fails to fetch the correct property or incorrectly uses a property

30%: **Complicated SPARQL**:
Fails to write complex SPARQL to fetch results.

15%: **Not enough exploration:**
Insufficient exploration within limit of actions allowed.

10%: **Inaccurate semantic parsing**:
LLM injecting an extra clause.

5%: **Formatting issues**

SPINACH Deployed on Wikidata Forum

https://m.wikidata.org/wiki/Wikidata:Request_a_query

- 22,000+ conversations in the wild: all real and hard queries!
- 198 randomly selected conversations
  - Success rate: 78% (154 cases) A higher success rate in practice!
  - Failures: 22% (44 cases)
    - 50% (22 cases) similar to queries in the dataset
    - 50% (22 cases) are not similar:
      underspecified queries, query correction/modification,
      string manipulation

Outline

- Data Structures: Graphs and Tables
- Graph QA with SPINACH
  - Challenges and Prior Approaches
  - The Agentic Approach
  - Dataset and Evaluation





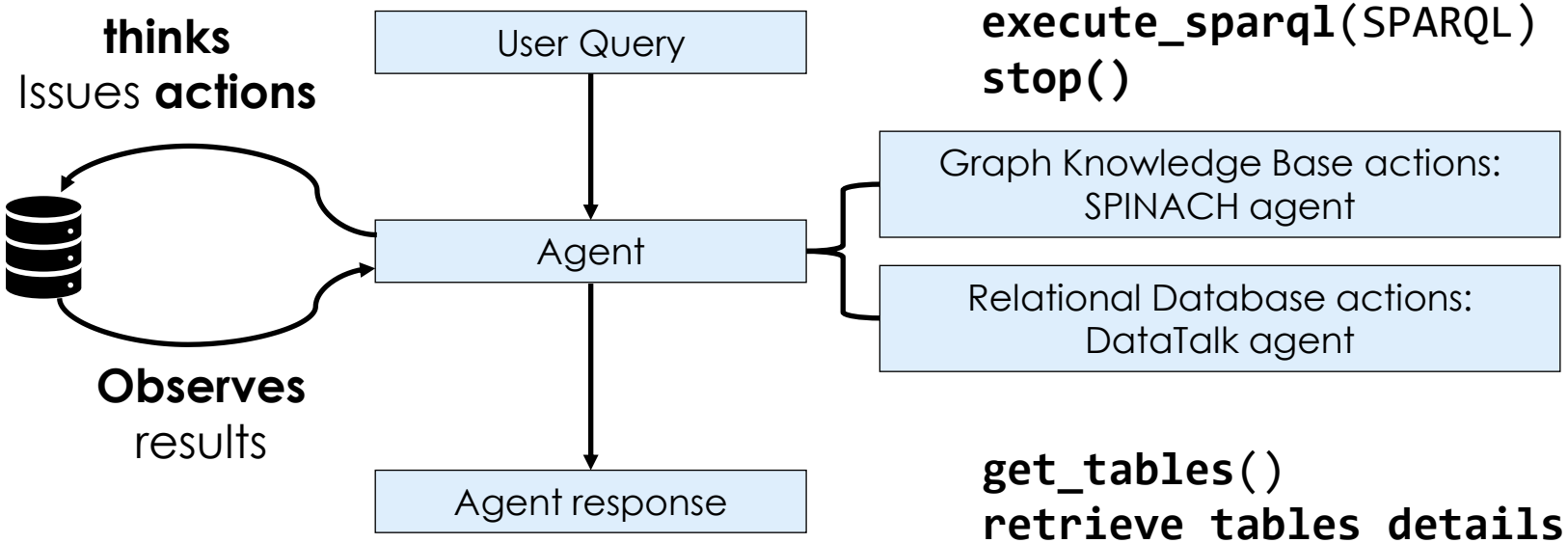
- **SQL QA with DataTalk**

- Guest Lecture: Big Local News on DataTalk (15 min)

From SPINACH to DataTalk

- **SPINACH: question answering over a knowledge graph**
  - Find entities and properties; build and execute SPARQL
- **DataTalk: question answering over relational tables**
  - Find tables, columns and join keys; build and execute SQL
- **The same agentic loop**
  - Inspect the data, write a query, execute, observe and revise

From SPARQL Actions to SQL Actions




```
search_wikidata(string)
get_wikidata_entry(QID)
get_property_examples(PID)
execute_sparql(SPARQL)
stop()
```

```
get_tables()
retrieve_tables_details([table1, table2, …])
execute_SQL(SQLquery)
entity_linking(search_str, [column1, column2, …])
location_linking(search_str)
stop()
```

The DataTalk Agent – Expert Instructions

execute_sql(



```sql
SELECT p."CMTE_ID", p."name", s."TTL_RECEIPTS", s."CVG_END_DT", m."CMTE_FILING_FREQ"
FROM "pac_details" p
JOIN "pac_and_party_summary_2023_2024" s ON p."CMTE_ID" = s."CMTE_ID"
JOIN "committee_master_2023_2024" m ON p."CMTE_ID" = m."CMTE_ID"
WHERE 'cryptocurrency' = ANY(p."industry")
ORDER BY s."TTL_RECEIPTS" DESC
LIMIT 10;
```

)

Also Shows specific instructions related to a SQL after the agent calls "execute_sql" in Observation

NEW!

- When using `pac_and_party_summary_2023_2024` about money raised, exclude committees whose IDs start with C9 because C9 committees are mostly only useful when analyzing expenditures. Apperance of them here likely correspond to errenous filings.
- When using `pac_and_party_summary_2023_2024`, include \"CMTE_FILING_FREQ\" and \"CVG_END_DT\" columns to qualify results.
- …

Guest Lecture

DataTalk & Role of Expert Knowledge

Big Local News

15 minutes

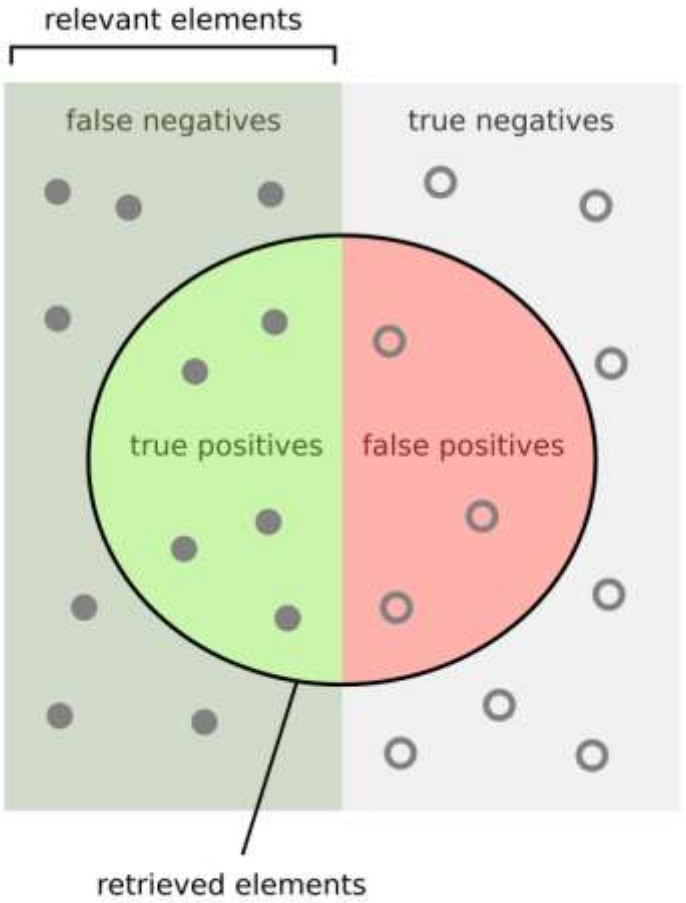
Appendix

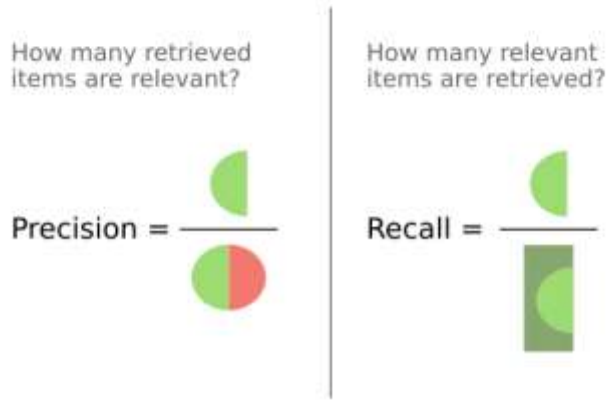
Calculating F1 for Tables

The F1 metric

$$F_1 = \frac{2}{\mathrm{recall}^{-1} + \mathrm{precision}^{-1}} = 2\frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}} = \frac{2\mathrm{TP}}{2\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}.$$

TP: True Positive
FP: False Positive
FN: False Negative





But: traditional EM & F1does not work



Predicted

Minnehaha County	**206,930**
Pennington County	**115,903**
Lincoln County	**73,238**

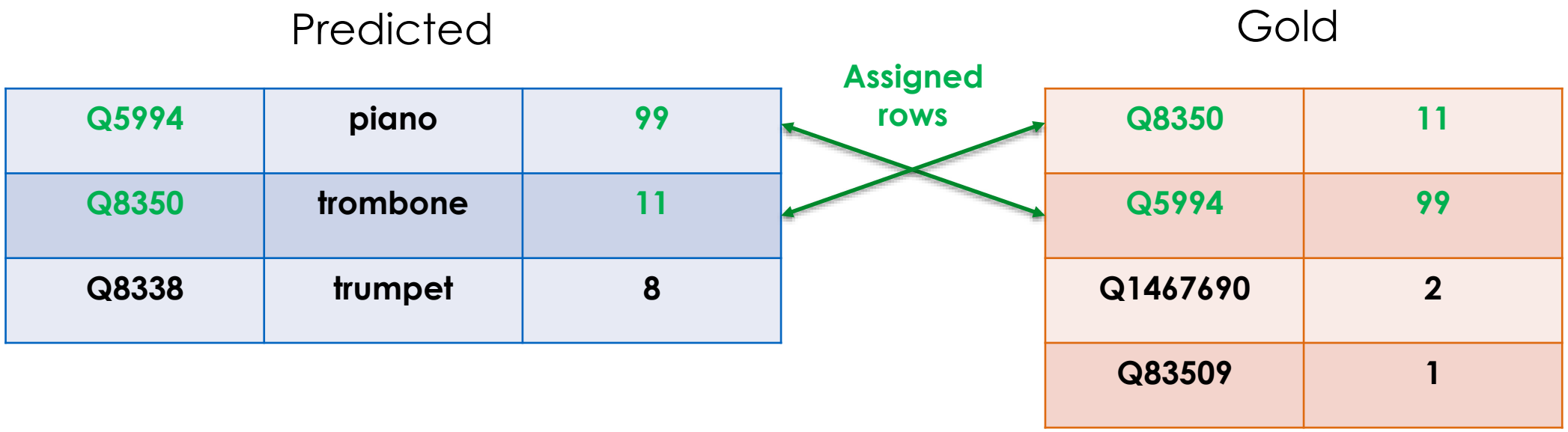
Gold

Minnehaha County
Pennington County
Lincoln County

Question: Do we want to penalize this column?

New Metric: **row-major** EM & F1

1. Assignment of rows



Step 1: Run an **assignment algorithm** to maximize **total overlap** (recall)

Without penalizing extra columns in prediction

y_i a row in gold

y'_i assigned row of y_i in prediction

$$\text{recall}(y_i, y'_j) = \frac{|y_i \cap y'_j|}{|y_i|}$$

Matching rows with 0 recall is not allowed

New Metric: **row-major** EM & F1

2. True positives

Predicted

Q5994	piano	99
Q8350	trombone	11
Q8338	trumpet	8

Assigned rows

Gold

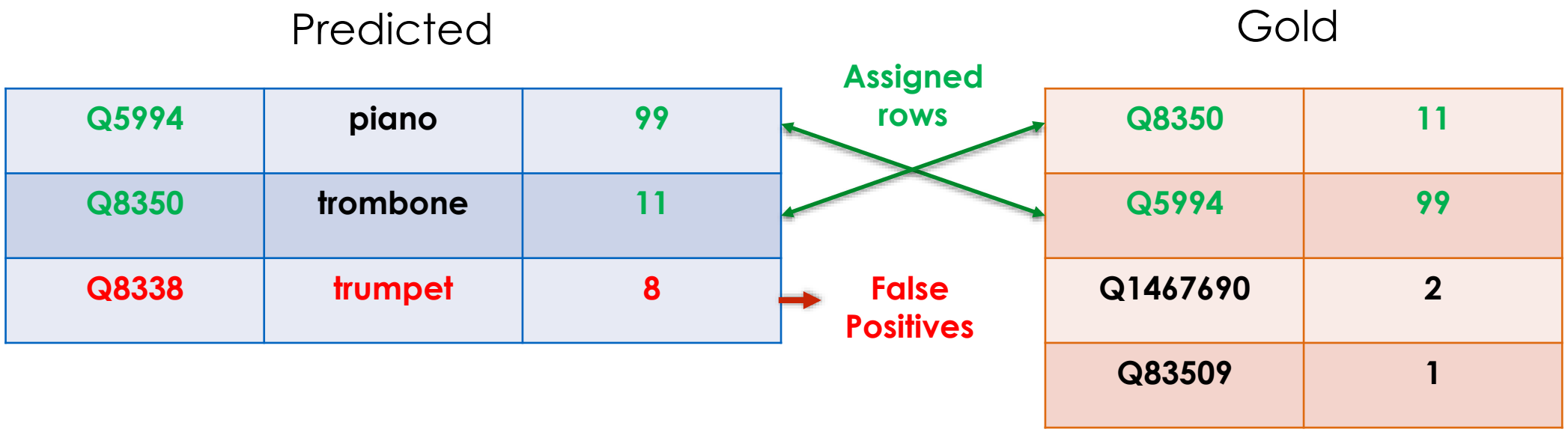
Q8350	11
Q5994	99
Q1467690	2
Q83509	1

Step 2: Calculate **true positives** as sum of recalls in assigned rows

$$tp = \sum_{(i,j)\in A(\mathbf{y},\mathbf{y}')} \text{recall}(y_i, y'_j) \qquad = 2$$

New Metric: **row-major** EM & F1

3. false positives



Step 3: Each unassigned row in prediction counts as a **false positive**

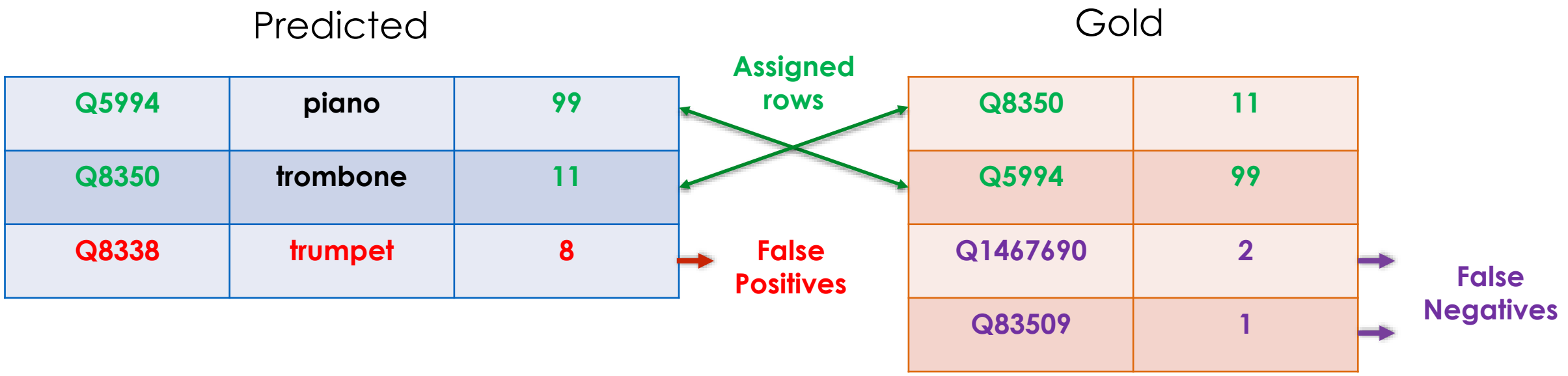
n': number of rows in prediction

r: number of rows in matching

$$fp = n' - r \qquad = 1$$

New Metric: **row-major** EM & F1

4. false negatives



Step 4: Each unassigned row in gold counts as a **false negative**

plus sum of 1– recall in the matching (0 in this case)

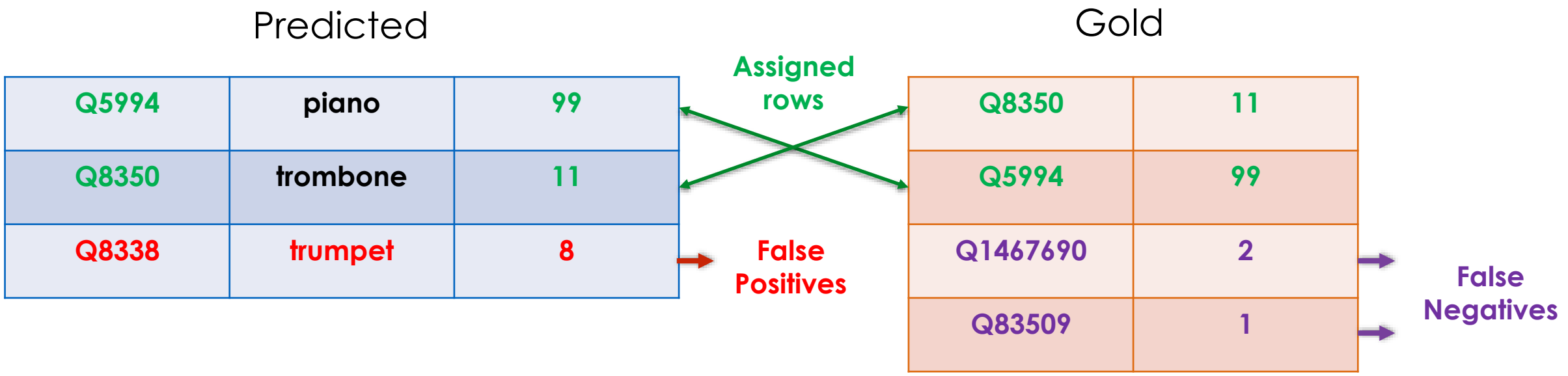
n: number of rows in gold

r: number of rows in matching

$$fn = n - r + \sum_{(i,j)\in A(\mathbf{y},\mathbf{y}')} 1 - \text{recall}(y_i, y'_j) \quad = 2$$

New Metric: **row-major** EM & F1

5. F1



Step 5: Calculate F1 with the usual formula

$$F_1 = \frac{2tp}{2tp+fn+fp} = 0.66$$